

Universitatea din Pitești
Facultatea de Electronică și Calculatoare

Etapele și riscurile unui proces software

Lect.univ.dr. Adrian Țurcanu
Lect.univ.dr. Cristina Tudose

Cuprins

- 1 Introducere
- 2 Specificare software
- 3 Dezvoltare software
- 4 Validare software
- 5 Evoluție Software
- 6 Riscurile unui proces software

Proces software

- Reamintim că, un proces software presupune 4 categorii de activități:
 - **Specificare software**: se definește funcționalitatea sistemului software și constrângerile cu care operează.
 - **Dezvoltare software** (Design și Implementare): se produce un sistem software care satisface specificațiile.
 - **Validare software**: se validează sistemul software pentru a se asigura că aceste corespunde cerințelor clientului.
 - **Evoluție software**: sistemul evoluează pentru a satisface noi cerințe.
- Fiecare dintre acestea, presupune parcurgerea unor etape pe care le prezentăm în continuare.

Specificare software - etape

- **Studiu de fezabilitate:** se evaluează dacă sistemul propus va fi eficient din punct de vedere al costurilor pentru satisfacerea cerințelor clientului și dacă poate fi dezvoltat ținând cont de limitele bugetului alocat.
- **Extragerea și analiza cerințelor:** se stabilesc cerințele prin observația unor sisteme existente sau discuții cu clientul/utilizatorii; poate presupune dezvoltarea unor modele și prototipuri care să ajute la înțelegerea sistemului.
- **Specificarea cerințelor:** informațiile colectate în faza de analiză sunt sintetizate într-un document care definește un set de cerințe ale sistemului.
- **Validarea cerințelor:** se verifică cerințele din punctul de vedere al realismului, consistenței și completitudinii; se corectează erorile în documentul de cerințe.

Specificare software

- Scopul parcurgerii acestor etape este producerea a două documente:
 - **Documentul Cerințelor Utilizatorului** (User Requirements Document - **URD**).
 - **Documentul Cerințelor Software** (Software Requirements Document - **SRD**)
- Cerințele descriu sistemul din punctul de vedere al utilizatorului vizând aspecte ca: funcționarea sistemului, performanța, securitatea sau interfața cu utilizatorul.
- SRD se obține în urma unei analize de specialitate a URD și conține o descriere completă a cerințelor produsului software.
- SRD stă la baza contractului dintre client și echipa de dezvoltare reprezentând o bază pentru estimarea costurilor și a planificării.

Dezvoltarea software

- Presupune o serie de activități legate de:
 - **Design**: proces iterativ care presupune obținerea unei descrieri a structurii sistemului software ce urmează a fi implementat, a datelor care fac parte din sistem, a interfețelor dintre componente și uneori a algoritmilor folosiți.
 - **Implementare și testarea primară**: urmează după design și presupune dezvoltarea unui program care implementează sistemul precum și o testare primară efectuată de programatori.

Activități ale procesului de desing

- **Design Arhitectural**: pornind de la SRD sunt identificate subsistemele și relațiile dintre ele.
- **Specificație Abstractă**: pentru fiecare subsistem este produsă o specificație abstractă a serviciilor și constrângerilor sub care operează.
- **Designul Interfeței**: interfata fiecarui subsistem este concepută și documentată.
- **Designul Componentelor**: serviciile sunt alocate diferitelor componente și sunt concepute interfetele dintre acestea.
- **Designul Bazelor de Date**: sunt proiectate bazele de date folosite.
- **Designul Algoritmilor**: sunt descriși algoritmi ce urmează a fi implementați.

Obiective ale procesului de design

- Scopul parcurgerii activităților procesului de design este producerea unor documente:
 - **Documentul de Proiectare Arhitecturală(ADD)**: descriere a sistemului în care trebuie să se regăsească toate cerințele acestuia.
 - **Documentul de Proiectare în Detaliu (DDD)**: descriere a sistemului ca o mulțime de componente direct implementabile și a funcțiilor pe care trebuie să le realizeze fiecare componentă.

Design - abordare în practică

În cazul multor proiecte, activitatea de design este un proces dinamic intercalat cu procesul de implementare.

- Se pornește cu un set de cerințe, de obicei în limbaj natural, care stau la baza unui design informal.
- Se începe implementarea, designul fiind modificat pe măsură ce implementarea evoluează.
- Deoarece nu există un control al schimbărilor, la finalizarea implementării, designul este schimbat într-o asemenea măsură încât documentul original reprezintă o descriere incorectă sau incompletă a sistemului.

În consecință, pentru o mai bună organizare și documentare, s-au elaborat metode structurate de design.

Metode structurate de design

- **metode orientate pe funcționalitate**: încearcă să identifice componentele funcționale de bază ale sistemului: Structured Analysis, Coad-Yourdan, Merise, SSADM.
- **metode orientate pe obiect**: Booch, OMT (Object Modeling Technique) (J. Rumbaugh), OOSE (Object Oriented Software Engineering) sau Objectory (I. Jacobson).
- toate metodele au fost unificate prin **UML** (Unified Modeling Language) și Unified Process (Booch, Rumbaugh, Jacobson).

Metode structurate de design

- Fiecare metodă structurată include un **model de proces** și un **set de notații** pentru reprezentarea designului însoțite de un set de reguli pentru folosirea acestora.
- Folosirea unei metode structurate duce la descrierea și documentarea sistemului sub forma unor **modele grafice**; fiecare metodă are un utilitar CASE asociat.
- Astfel de metodele sunt folosite cu succes în proiecte de mari dimensiuni, deoarece producerea unei documentații standard pentru desing duce la o reducere a costurilor.
- Alegerea unei metode este dependentă de domeniul aplicațiilor pentru care aceasta este folosită.

Tipuri de modele

- **Modele structurale (arhitecturale)**: prezintă componentele sistemului și interacțiunile dintre acestea
- **Modele comportamentale**: descriu comportamentul de ansamblu al sistemului
 - **Modelul fluxului de date**: descrie fluxul de date între procese în cadrul sistemului (specifice sistemelor economice)
 - **Modelul mașinii cu stări**: descrie comportamentul sistemului ca răspuns la evenimente externe sau interne (specifice sistemelor interactive)

Tipuri de modele

- **Modele de date:** definesc forma logică a datelor procesate de către sistem
 - **Modelul entitate-legătură:** descrie entitățile din sistem și relațiile dintre acestea; este folosit frecvent pentru descrierea bazelor de date.
- **Modele obiectuale:** identifică obiectele, clasele din care fac parte și interacțiunile dintre acestea
 - **Modele structurale:** identifică care sunt clasele și legăturile dintre acestea (diagrame de clase, ierarhii de moștenire)
 - **Modele comportamentale:** arată modul de utilizare a operațiilor din diagrama de clase (diagrame de interacțiune (secvență sau colaborare)).

Implementare și testare primară

- Urmează finalizării desingului și presupune transformarea modelului arhitectural în program precum și o testare care vizează eventualele erori de programare.
- Implementarea se realizează, de obicei modular, urmând ca apoi modulele să fie integrate treptat, până la nivel de sistem.
- Se verifică comunicarea și interacțiunea dintre componentele integrate.
- Implementarea reprezintă doar 15 – 20% din efortul total de dezvoltarea a unui sistem software.

Validarea software

- Verificarea și validarea au ca scop să arăte că sistemul este conform cu specificația și satisface așteptările clientului.
- Implică proceduri de verificare precum **inspecții** și **revederi** care au loc în fiecare fază a procesului de dezvoltare, de la definirea cerințelor la implementare.
- Activitatea cea mai importantă și mai costisitoare a fazei de validare are loc după faza de implementare, atunci când este **testat** sistemul operațional.
- Proiectarea și dezvoltarea modulară a sistemelor de mari dimensiuni implică și testarea acestora pe module, pe măsura ce acestea sunt integrate în sistem.

Metode de testare

- **Testare la nivel de unitate (unit testing)**: este testată funcționarea corectă a componentelor individuale.
- **Testare la nivel de modul (module testing)**: este testată funcționarea modulelor formate din componente dependente (clase, pachete).

Aceste metode de testare sunt aplicate, în general de programatori.

Metode de testare

- **Testare la nivel de sub-sistem (sub-sistem testing)**: sunt testate sub-sisteme obținute prin integrarea modulelor cu accent pe interfețele dintre module.
- **Testare la nivel de sistem (system testing)**: este testat sistemul în ansamblu pentru a se identifica erorile cauzate de interacțiunea dintre subsisteme. În plus, se validează că sistemul îndeplinește cerințele funcționale și nefuncționale.
- **Testare pentru acceptare (acceptance testing)**: sistemul este testat cu date furnizate de către client. Poate scoate în evidență probleme legate de definirea cerințelor, funcționalități incomplet sau incorect implementate sau probleme de performanța ale sistemului.

Alpha testing vs. Beta testing

- Testarea pentru acceptare este numita și **testare alfa (alpha testing)** și este un proces care continuă până când clientul și dezvoltatorul cad de acord că sistemul reprezintă o implementare acceptabilă a cerințelor.
- Testarea alfa se aplică în cazul sistemelor personalizate.
- Când un sistem este comercializat ca un produs software generalizat, adeseori se folosește un proces de testare numit **testare beta (beta testing)**.
- Testarea beta presupune livrarea unui sistem unui număr de potențiali clienți care au fost de acord să utilizeze sistemul și să raporteze potențiale erori.

Evoluție software

- reprezintă procesul de schimbare a produsului software după ce acesta a fost dat în funcțiune
- presupune atât corectarea unor erori de funcționare cât și adăugarea unor noi funcționalități
- în cazul unor produse software personalizate schimbările sunt solicitate de client/utilizator
- schimbările de funcționalitate pot fi foarte costisitoare; cu toate acestea sunt, în general, mult mai ieftine decât schimbările corespunzătoare de hardware

Dezvoltare vs. Mentenanță

- În primele decade ale evoluției sistemelor software exista o delimitare clară între dezvoltare și mentenanță.
- Dezvoltarea presupune crearea unui sistem nou pornind de la un concept inițial.
- Mentenanța, pe de alta parte, presupune modificari ale unui sistem existent și, deși este adesea foarte costisitoare, este considerată de către programatori dificilă și mai puțin interesantă decât dezvoltarea.

Dezvoltare vs. Mentenanță

- Cu timpul însă, demarcația între dezvoltare și mentenanță a devenit din ce în ce mai irelevantă în contextul unui număr din ce în ce mai redus al sistemelor complet noi.
- În consecință, în locul celor doua procese separate, se consideră un singur proces de evoluție, în care sistemul este schimbat continuu, pe parcursul folosirii acestuia, în funcție de schimbări ale cerințelor și nevoile utilizatorilor.

Riscurile unui proces software

Sunt 4 tipuri principale de factori de risc:

- de experiență:
 - lipsa de experiență sau calificare a managerului de proiect și a echipei
 - lipsa implicării acestora în dezvoltarea unor proiecte similare
- de planificare
 - estimarea incorectă a resurselor umane necesare în fiecare etapă a proiectului
 - estimarea incorectă a intervalelor de timp necesare pentru diferite activități, inclusiv pentru livrarea produsului
 - repartizarea ambiguă, fără temei sau disproporționată a responsabilităților

Riscurile unui proces software

- tehnologici
 - noutatea tehnologică
 - alegerea unor metode de dezvoltare noi sau nepotrivite
 - folosirea unor instrumente de dezvoltare ineficiente sau inadecvate
- externi
 - cerințe ambigue ale clienților/utilizatorilor și calitatea specificațiilor aferente
 - slaba calitate a definirii și a stabilității interfețelor externe
 - slaba calitate sau lipsa de disponibilitate a sistemelor externe